Unit 4: Inheritance, Polymorphism, Interface and Packages 8 hours

4.1 Inheritance
- 4.1.1 Using Extends
- 4.1.2 Member access
- 4.1.3 Super classes and sub classes
- 4.1.4 Single Inheritance
- 4.1.5 Multi-level Inheritance
- 4.1.6 Hierarchy Inheritance

4.2 Constructor and inheritance
4.3 Polymorphism-Method overriding
4.4 Abstract class
4.5 Using final keyword
4.6 Interface
- 4.6.1 Creating and implementing an interface
- 4.6.2 Extending interface

4.7 Packages
- 4.7.1 Creating Packages
- 4.7.2 Accessing a Package

4.1 Inheritance

Inheritance is a mechanism where one class (subclass/child) acquires the properties and behaviors (fields and methods) of another class (superclass/parent).

It supports code reusability, extensibility, and hierarchical relationships.

4.1.1 Using extends

The extends keyword is used to create a child class from a parent class.

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
```

```
        }
}
```

Here, Dog inherits the method eat().

4.1.2 Member Access

Member access rules:

Member Type	Same Class	Subclass (same package)	Subclass (different package)	Non-subclass
Public	✔	✔	✔	✔
protected	✔	✔	✔ (via inheritance)	✗
Default	✔	✔	✗	✗
Private	✔	✗	✗	✗

Example:

```
class A {
    private int x = 10;
    public int y = 20;
    protected int z = 30;

}

class B extends A {
    void display() {
        // x not accessible
        System.out.println(y);          // public
        System.out.println(z);          // protected
    }
```

```
}
```

4.1.3 Superclasses and Subclasses

- Superclass: Class whose features are inherited.
- Subclass: Class that inherits features and can add more or override.

```
class Vehicle { ... }                // Superclass
class Car extends Vehicle { ... } // Subclass
```

A subclass has:

- All non-private members of the superclass
- Can add its own fields/methods
- Can override methods

4.1.4 Single Inheritance

One superclass → one subclass.

```
class A { }
class B extends A { }
```

4.1.5 Multilevel Inheritance

Chain of inheritance: A → B → C

```
class A { }
class B extends A { } class C extends B { }
```

4.1.6 Hierarchical Inheritance

One parent → multiple children.

```
class A { }
class B extends A { } class C extends A { }
```

4.2 Constructor and Inheritance

Constructors are not inherited, but a subclass always calls the superclass constructor (implicitly or explicitly).

- Java automatically calls super() (default constructor) if not written.
- To call a parameterized parent constructor:

```
class A {
    A(int x) {
        System.out.println("A's constructor");
    }
}

class B extends A {
B() {
        super(10);  // calling parent constructor
        System.out.println("B's constructor");
    }
}
```

4.3 Polymorphism – Method Overriding

Polymorphism = "one interface, many forms."

Method Overriding

Subclass provides a specific implementation for a method already defined in the parent class.

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }

}

class Dog extends Animal {
```

```
@Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Dynamic Method Dispatch

```
Animal a = new Dog();
a.sound();          // Dog barks
```

4.4 Abstract Class

An abstract class is a class declared with abstract keyword.

- Cannot be instantiated.
- Can contain abstract methods (without body).
- Subclasses must override abstract methods.

Example:

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}
```

4.5 Using final Keyword

Usage	Meaning

final variable	Value cannot change (constant)
final method	Cannot be overridden
final class	Cannot be inherited

Examples:

```
final int a = 10;

final class A { } // cannot extend

class B {
    final void show() { }
}
```

4.6 Interface

An interface is a fully abstract type that contains:

- public abstract methods (implicitly)
- public static final constants
- Default and static methods (Java 8+)

4.6.1 Creating and Implementing an Interface

```
interface Animal {
    void sound();          // abstract
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}
```

A class uses implements to use an interface.

4.6.2 Extending Interface

An interface can extend multiple interfaces.

```
interface A {
    void show();
}
interface B {
    void display();
}

interface C extends A, B { }
```

4.7 Packages

Packages are used to organize classes and avoid name conflicts.

- Built-in packages: java.util , java.io , etc.
- User-defined packages.

4.7.1 Creating Packages

```
package mypackage;

public class Test{
    public void how() {
        System.out.println("Hello from package");
    }
}
```

Save file in a folder named mypackage .

4.7.2 Accessing a Package

```
import mypackage.Test;

class Demo {
    public static void main(String[] args) {
        Test t = new Test();
        t.show();
    }
}
```